Aplikacje klienckie i wykorzystanie JDBC
Database applications and JDBC

dr hab. inż. Maciej Grzenda
M.Grzenda@mini.pw.edu.pl
http://www.mini.pw.edu.pl/~grzendam

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

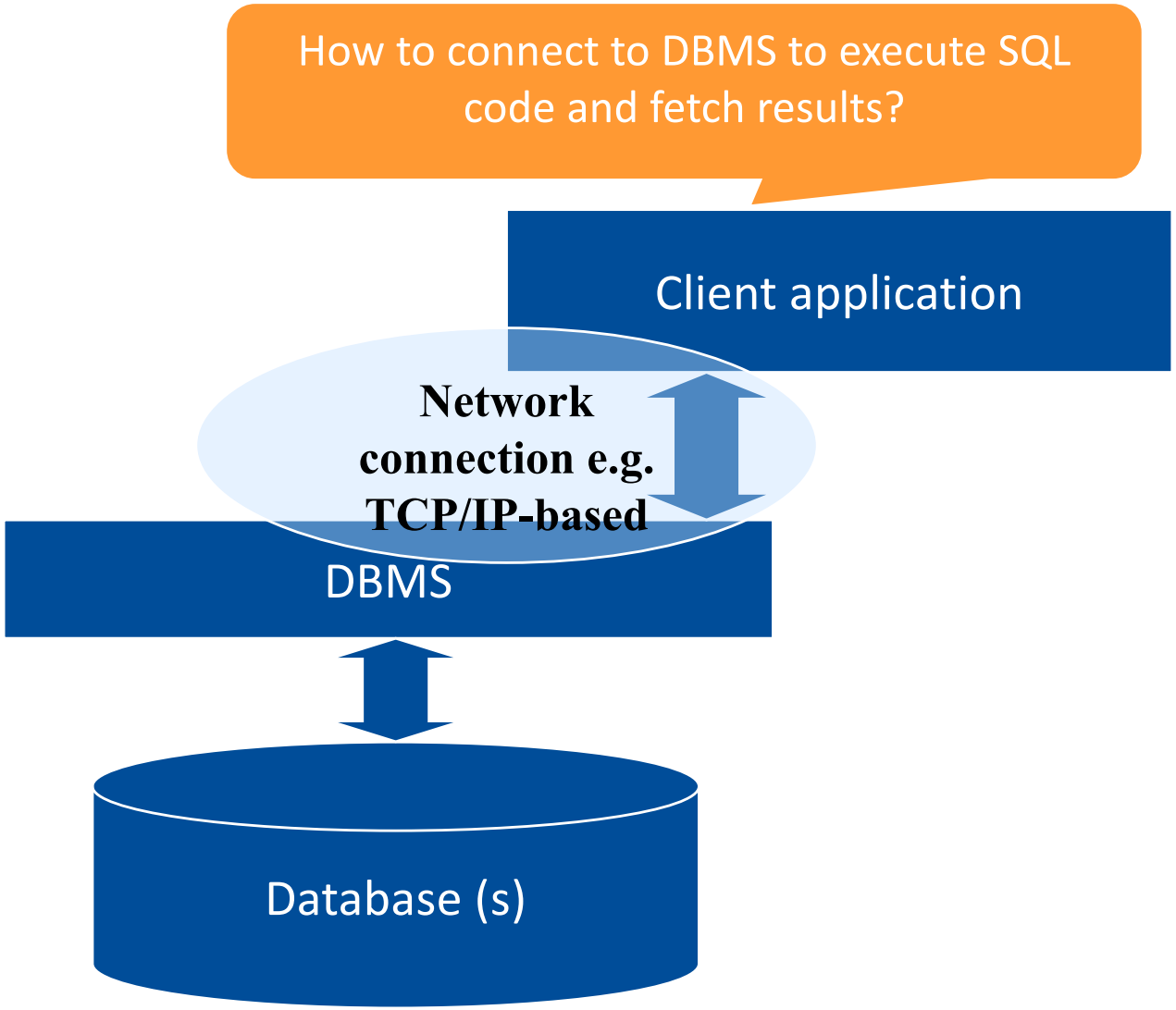
Warsaw University of Technology

MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Objectives

- Real life use of DBMS involves not only management tools, but even more importantly client applications
- Hence, the question arises how to develop an application which connects with a DBMS to:
  - execute SQL SELECT queries to fetch the data of interest to display it to the user, develop reports out of it or perform analytical processing
  - apply changes to the data and data model with DML and DDL statements
- JDBC is one of leading standards making the development of client applications in Java possible

Database and DBMS: a typical layout



JDBC

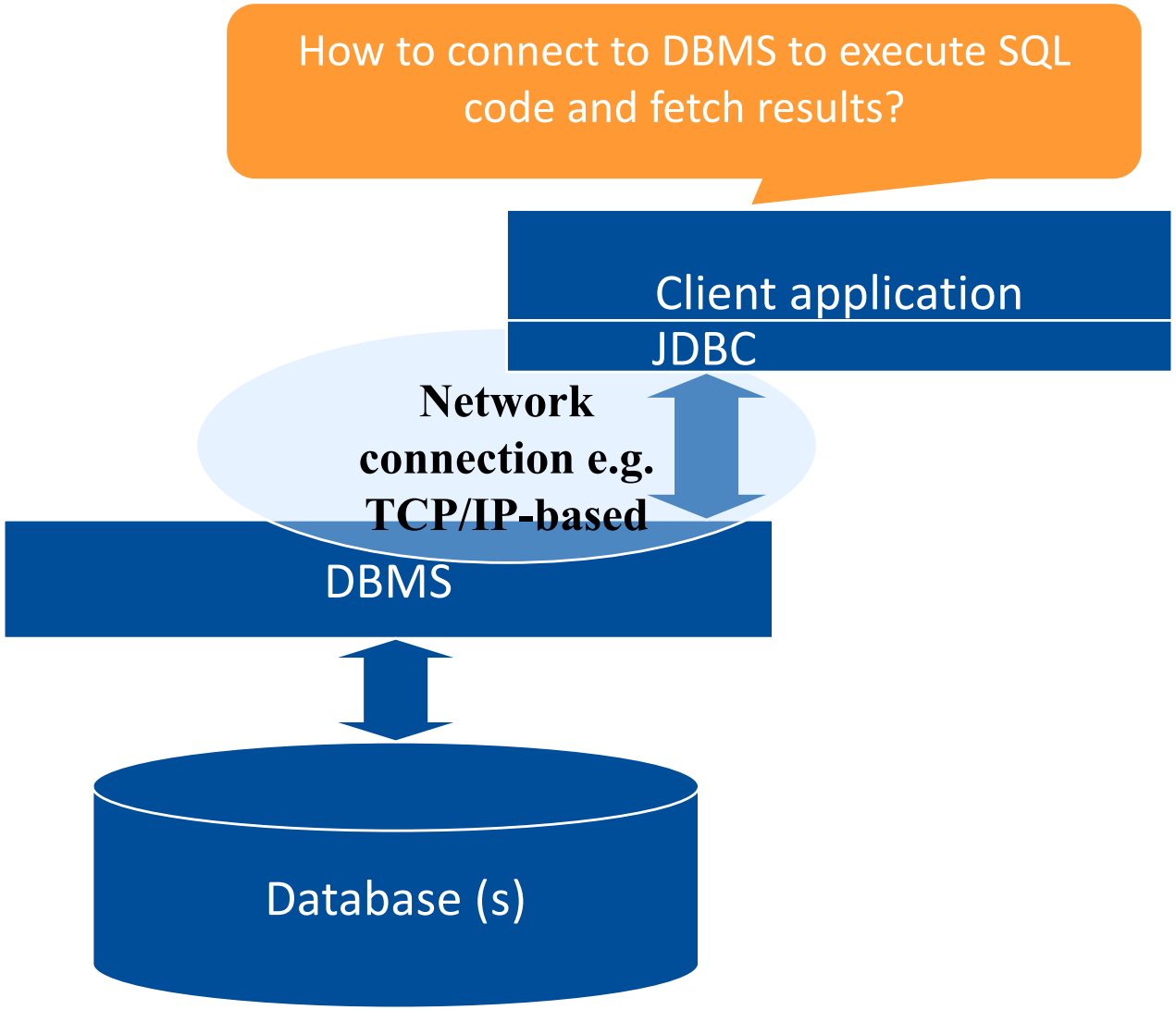
- Stands for Java Database Connectivity
- Provides data access from the Java programming language to different data sources including:
  - Relational databases,
  - Spreadsheets,
  - Flat files.
- JDBC API is composed of:
  - `java.sql` package, which provides core classes and interfaces
  - `javax.sql` package which contains server-side extensions.

JDBC API: resources

- JDBC has a long history and is not evolving much as not much has to change
- Sample description of java.sql package:

  https://docs.oracle.com/en/java/javase/14/docs/api/java.sql/java/sql/package-summary.html
- Sample description of javax.sql package i.e. server-side data source access:
  - https://download.java.net/java/GA/jdk14/docs/api/java.sql/javax/sql/package-summary.html
- Some tutorials:
  - https://www.javaworld.com/article/3388036/what-is-jdbc-introduction-to-java-database-connectivity.html
  - https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html

Database and DBMS: a typical layout



Why JDBC API?

- *The JDBC API is* ***a standard API for tool/database developers and makes it possible to write industrial strength database applications using an all-Java API****.*
- ***The JDBC API makes it easy to send SQL statements to relational database systems and supports all dialects of SQL****. But the JDBC 2.0 API goes beyond SQL, also making it possible to interact with other kinds of data sources, such as files containing tabular data.*
- ***The value of the JDBC API is that an application can access virtually any data source and run on any platform with a Java Virtual Machine****.*

Based on JDBC Technology Guide available at
http://java.sun.com/javase/6/docs/technotes/guides/jdbc/getstart/GettingStartedTOC.fm.html
The reasons cited above remain true also nowadays.

Establishing a connection – the first option

- The first option is to load a specified JDBC driver in order to establish a connection
- Advantages:
  - The same API can be used to access different databases
- Disadvantages:
  - JDBC driver name hard coded in the application
  - It takes time to establish and close a connection; whenever possible connection pooling should be applied.

Database connection

Sample code - Java

```
Connection con = null;
try {
        Class.forName("oracle.jdbc.driver.OracleDriver");



con =
   DriverManager.getConnection("jdbc:oracle:thin:@localh
   ost:1521:ORCL", "hr", "hrpass");

   Statement statement = con.createStatement();
   ResultSet rs = statement.executeQuery("SELECT * FROM
   Jobs");
   while ( rs.next() ) {
         // get the title_name,
        // which is a String
         out.println(rs.getString("title_name"));
......
rs.close();
```

JDBC Driver is loaded

Connection is established (if everything goes fine)

Query is executed through a connection

Results are accessed via a ResultSet object

Establishing a connection – sample code – part I

```
import java.sql.*;
public class ConnectionTester {
    public ConnectionTester() {
    }
    public static void main(String[] args) {
       …

         Connection con = null;
             try {
       Class.forName("oracle.jdbc.driver.OracleDriver");
  con =
  DriverManager.getConnection("jdbc:oracle:thin:@localho
  st:1521:ORCL", "hr", "hrpass");

  Statement statement = con.createStatement();
   ResultSet rs = statement.executeQuery("SELECT * FROM
  Jobs");
```

Establishing a connection – sample code – part II

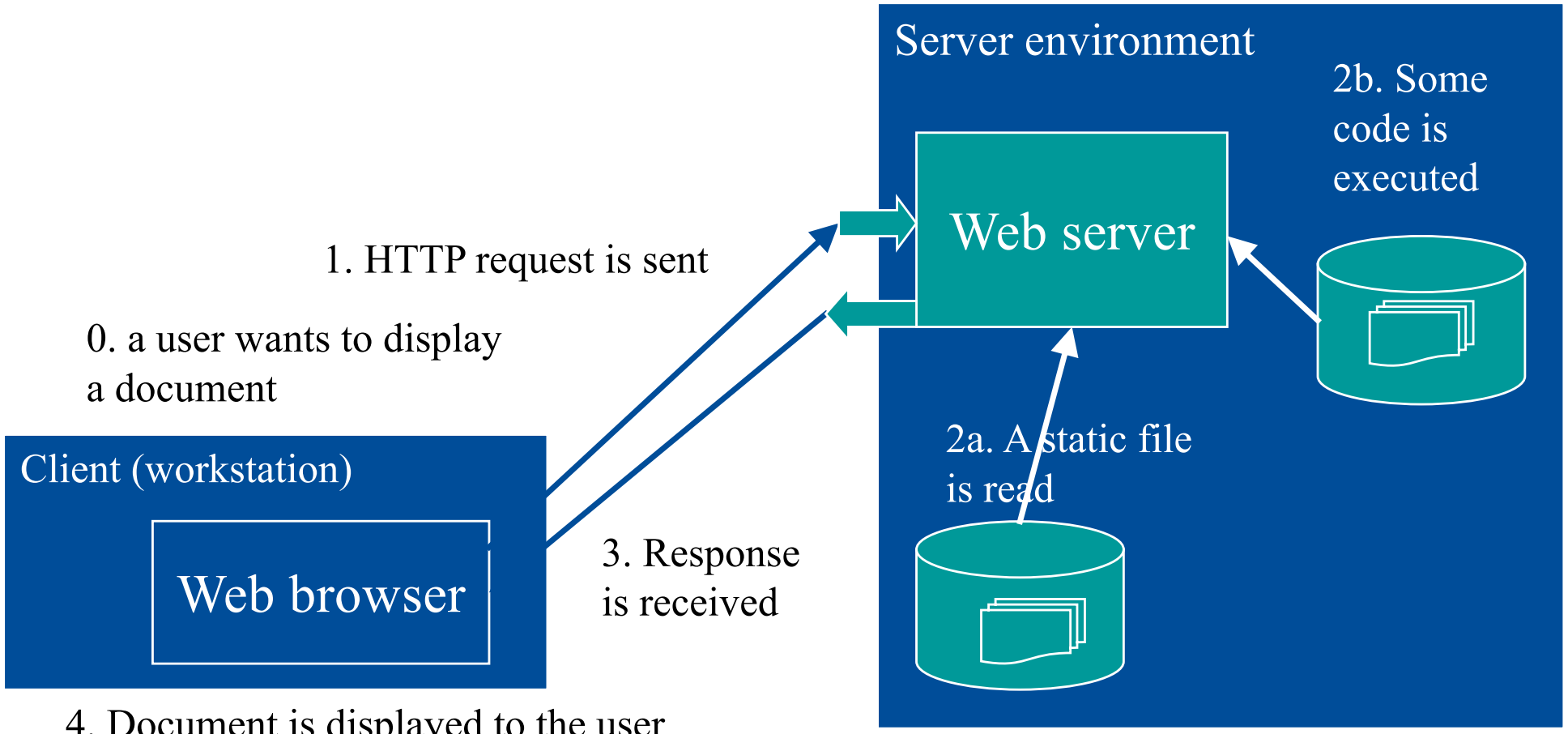
```
 while (rs.next()) {
                    System.out.println("Job:
   "+rs.getString("Job_Title")+" with salary starting
   from "+rs.getString("Min_Salary"));
}
   statement.close();
    rs.close();
    con.close();

            } catch (ClassNotFoundException e) {
               e.printStackTrace();
         } catch (SQLException e) {
               e.printStackTrace();
         }

     }
}
```

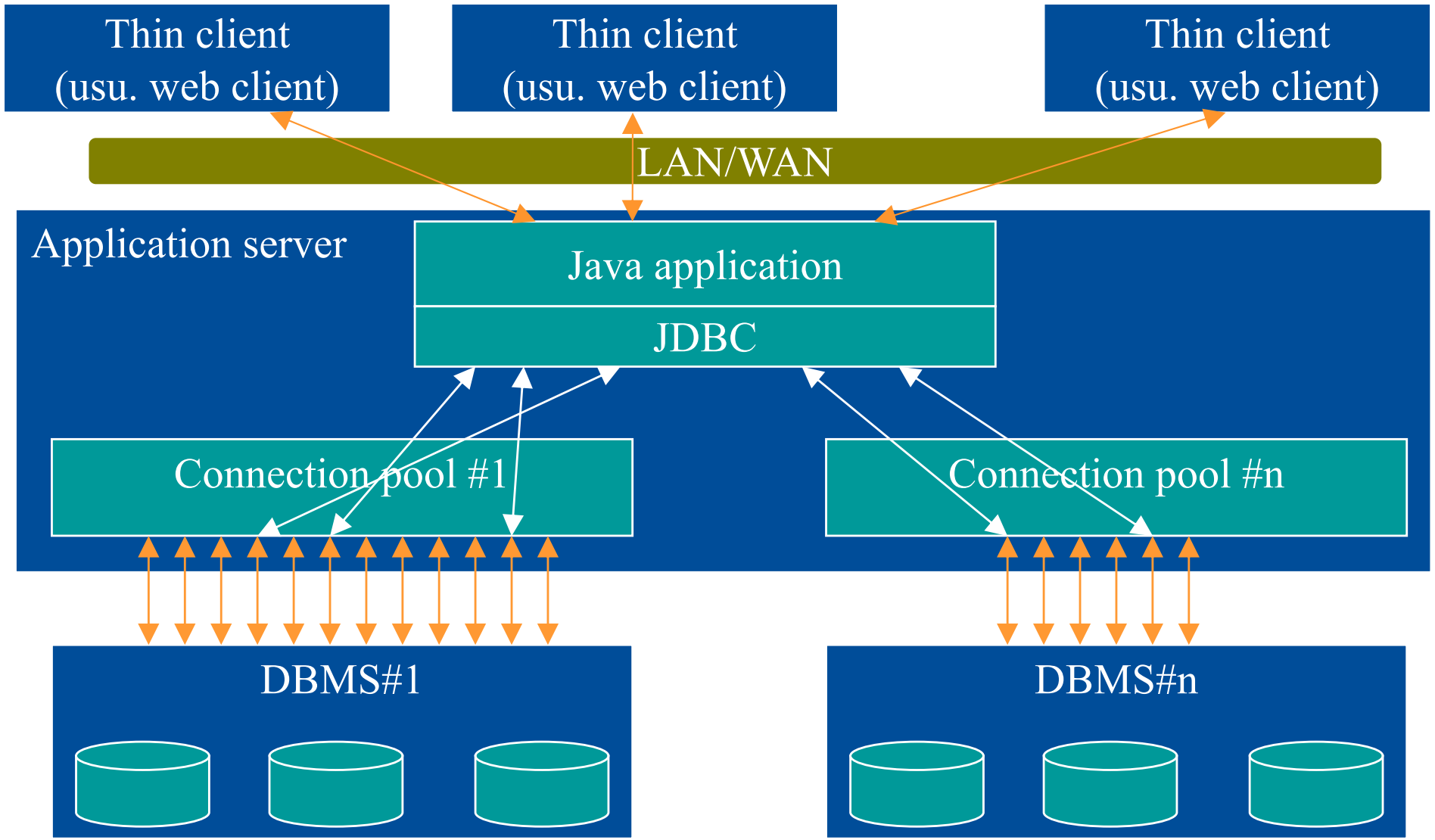
JDBC Oracle driver is used. Another driver would be needed to access another DBMS.

Dynamic documents and server-side applications



In this case documents are generated on-line

Connection pooling
Pule połączeń



Establishing a connection – the 2nd option

- The second option is to use a connection obtained from a data source
- Advantages:
  - The same API can be used to access different databases
  - The connection attributes including driver type and DBMS platform are not hard coded
  - Connection pool can be accessed
- Disadvantages:
  - Application server is required to host the Java application. This option can be used by server-side applications

Connection from a DataSource – part I

```
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import javax.naming.InitialContext;
import javax.naming.NamingException;
import javax.servlet.*;
import javax.servlet.http.*;

import javax.sql.DataSource;

public class testServlet extends HttpServlet {
    private static final String CONTENT_TYPE = "text/html";

    public void init(ServletConfig config) throws
   ServletException {
        super.init(config);
    }
```

Connection from a DataSource – part II

```
public void doGet(HttpServletRequest request,
                        HttpServletResponse response) throws
    ServletException, IOException
    {response.setContentType(CONTENT_TYPE);
         PrintWriter out = response.getWriter();
         out.println("<html>");
        Connection con = null;
          try {
                InitialContext ic=null;

              try {
                  ic = new InitialContext();
              } catch (NamingException e) {
                  …
              }
              DataSource ds=null;
              try {
                  ds = (DataSource) ic.lookup("jdbc/DBTestDS");
              } catch (NamingException e) {
                 …
              }
              con = ds.getConnection();
```

A reference to JNDI DataSource is made. Thus, the real configuration of a connection (the type and location of a DBMS, driver type, security credentials) is no longer hard coded.

Connection from a DataSource – part III

```
Statement statement = con.createStatement();
ResultSet rs = statement.executeQuery("SELECT * " + "FROM
   Jobs");

while (rs.next()) {
  out.println("<li> Job: "+rs.getString("Job_Title")+" with
   salary starting from "+rs.getString("Min_Salary"));
        }
             statement.close();
             rs.close();
               con.close();

        } catch (SQLException e) {
            e.printStackTrace();
        }
        out.println("</body></html>");
        out.close();
    }
}
```

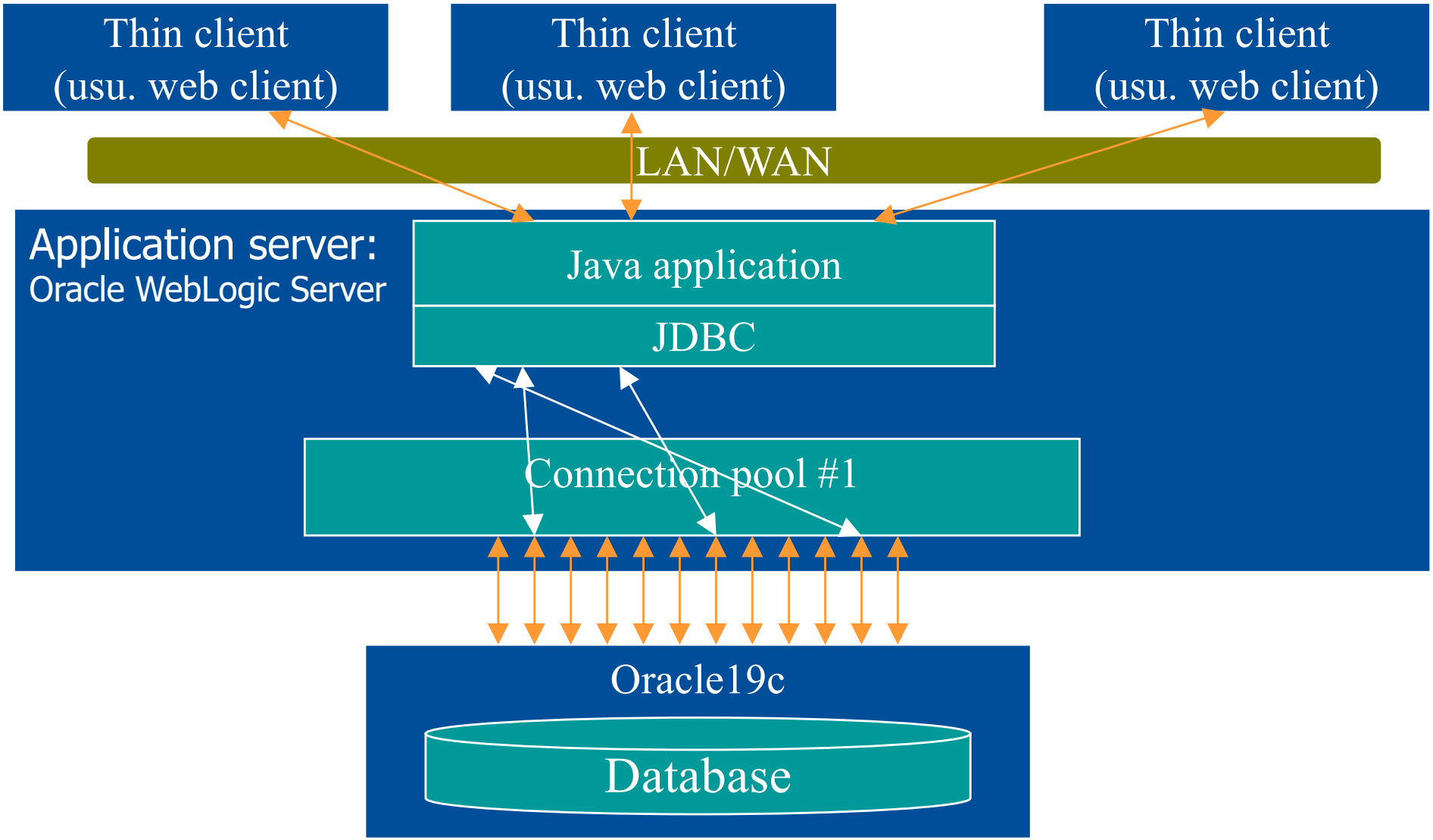
The rest of the code is quite similar to the previous example. The code is based on JavaServlet. In fact Java EE application server providing JNDI services is used in the example.

Data sources - defined

```
<?xml version = '1.0' encoding = 'windows-1250'?>
<data-sources xmlns:xsi="http://www.w3.org/2001/XMLSchema-
   instance"
   xsi:noNamespaceSchemaLocation="http://xmlns.oracle.com/orac
   leas/schema/data-sources-10_1.xsd"
   xmlns="http://xmlns.oracle.com/oracleas/schema">
    <connection-pool name="jdev-connection-pool-DBTest">
        <connection-factory factory-
   class="oracle.jdbc.pool.OracleDataSource" user="hr"
   password="->DataBase_User_iJr2Heyyw8WoyfiTIBHQSsZfBbm45XmM"
   url="jdbc:oracle:thin:@localhost:1521:ORCL"/>
    </connection-pool>

   <managed-data-source name="jdev-connection-managed-DBTest"
   jndi-name="jdbc/DBTestDS" connection-pool-name="jdev-
   connection-pool-DBTest"/>
     ...
</data-sources>
```

The content can be found in <ApplicationName>-data-sources.xml, which is a part of Java application configuration. The configuration is used by Java EE application server to establish data sources used by Java applications e.g. JavaServlet-based web applications.


Thin client
(usu. web client)
Thin client
(usu. web client)
Thin client
(usu. web client)
LAN/WAN
Application server:
Oracle WebLogic Server
Java application
JDBC
Connection pool #1
Oracle19c
Database

Connection pooling

- A pool of connections is created by an application server
- In JBDC, a DataSource object can be used to access such connections. In that case:
  - getConnection() usu. returns an existing connection,
  - close() releases a connection so that it can be reused.
- Existing connections to a DBMS can be reused instead of being closed
- This may result in dramatic performance improvements
- The application server maintains a pool of connections, as configured by the administrator
- Obtaining a connection through a DataSource is a suggested solution in multi-user enterprise applications

Whether a DataSource operates a connection pool or not depends on the developers implementing it. The DataSource implementations are provided by DBMS vendors and third-party vendors.

Data modifications - PreparedStatement

```
…
 con = ds.getConnection();

 PreparedStatement ps = con.prepareStatement(
    "UPDATE Jobs SET MIN_SALARY = ? WHERE
      JOB_Title = ?");

 ps.setInt(1,100);
 ps.setString(2,"Finance Manager");

 ps.executeUpdate();
 con.commit();
```

Whenever SQL Statement is supposed to be executed a number of times, PreparedStatement should be preferred. It creates a precompiled statement on the DBMS side, which allows to significantly reduced time required to execute a query. The solution can be used both for queries selecting and modifying the data. A regular Statement can be used otherwise.

SQL and SECURITY THREATS

Combining query text with user input I

Let us consider the following snippet:

```
Statement statement = con.createStatement();
String queryText = "select CustomerId,CompanyName,Phone from
       Customers where  CustomerID=";
String userInput = myInterface.getUserInput();

ResultSet rs = statement.executeQuery(queryText + "'" +
userInput + "'");
while (rs.next()) {
  System.out.println("Employee: " + rs.getString("CustomerId")
  + " " +
  rs.getString("CompanyName") +" " + rs.getString("Phone"));
}
```

myInterface.get UserInput() is our method that is supposed to provide us with the value typed in by a user. We expect something like ALFKI to be placed in the variable.

Next, we are going to show selected features of the customer to the user (here just illustrated with System.out. println() for simplicity). Hence, we execute a trivial query concatenating user input with query text. Any risks?

Combining query text with user input II

What if a user provided the following data:"ALFKI' union select FirstName,LastName,City+' '+Address from Employees union select '','' ?
This means:
userInput = "ALFKI' union select FirstName,LastName,City+' '+Address from Employees union select '','';

```
String queryText = "select CustomerId,CompanyName,Phone from
        Customers where        CustomerID=";
String userInput = myInterface.getUserInput();
ResultSet rs = statement.executeQuery(queryText + "'" + userInput +
"'");
while (rs.next()) {
 System.out.println("Employee: " + rs.getString("CustomerId") + " " +
    rs.getString("CompanyName") +" " + rs.getString("Phone"));
 }
```

Suddenly, the same code provides the user with the data of ALFKI customer, but also with home addresses of all employees of our company. This illustrates the problem of SQL code injection i.e. security threat based on extending SQL code with malicious phrases changing the original intention of the query.

Combining query text with user input III

SQL code injection can be attempted with any API (such as JDBC or ADO.NET) and any DBMS. It is even easier with web applicatins.
One of the ways to reduce the risk of code injection is to avoid concatenating user input with query code. Instead, parametrised queries such as done with PreparedStatement (in JDBC) or SQLCommand (in. NET) can be used to make it clear that a value rather than SQL code is supposed to be placed in a certain part of an SQL statement.

```
PreparedStatement ps = con.prepareStatement("select
CustomerId,CompanyName,Phone from Customers where CustomerID = ?");
ps.setString(1, userInput);

rs = ps.executeQuery();
   while (rs.next()) {
     System.out.println("Employee: " + rs.getString("CustomerId") + " "
     +rs.getString("CompanyName") +
     " " + rs.getString("Phone"));
}
```

This time, the output of the query is empty. The data of all employees is nor revealed. Why?

Discussion

- ResultSet objects have to be closed as soon as possible
- Connection objects have to be closed (or returned to connection pool) as soon as possible
- Hint:
  - Manage all the resources of this kind in an organised way e.g. via the same monitoring object throughout an entire application
  - Monitor the number of used database objects (result sets, connections,…) to avoid resource leak

try-with-resources

- Try-with-resources can be used to close resources that implement java.lang.AutoCloseable interface
- Example (adapted from https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html):

```
try (Statement stmt = con.createStatement()) {
     ResultSet rs = stmt.executeQuery(query);
     while (rs.next()) {
            String coffeeName = rs.getString("COF_NAME"); …
     }
     } catch (SQLException e) {
     JDBCTutorialUtilities.printSQLException(e); }
```

- In such cases, Statement and Results will be closed. In try-with-resources, any catch or finally block is executed after the resources declared have been closed.

Note that: ResultSet and Statement will be closed. However, handling Connection (which also implements the java.lang.AutoCloseable interface) in the same way can be risky, as the JDBC standard e.g. in https://docs.oracle.com/en/java/javase/23/docs/api/java.sql/java/sql/Connection.html#close() states: „It is strongly recommended that an application explicitly commits or rolls back an active transaction prior to calling the close method. If the close method is called and there is an active transaction, the results are implementation-defined.”. If we declare Connection within try-with-catch it can be closed before we resolve the status of transaction

Summary

- Java is frequently used for the development of database applications
- JDBC is the core standard enabling Java developers *inter alia* to:
  - Connect to a DBMS
  - Submit SQL SELECT queries
  - Submit DML and DDL statements such as SQL INSERT statements
  - Commit (and rollback) transactions
- More recently, Java Persistence API (Jakarta Persistence API) provides object-relational mapping solutions for Java.
- Still, JDBC remains the core standard on top of which additional layers can be built.

Warsaw University of Technology

MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.